FUNDAMENTOS DE GIT: REPOSITORIOS, CAMBIOS Y CONFIRMACIONES

Institución educativa: Fundación universitaria Agraria de Colombia

Programa académico: Medicina Veterinaria

Asignatura: Introducción a la ciencia de datos

Título del trabajo: Tutorial de Git — Capítulo 2: Fundamentos de Git

Estudiante: July Manuela Romero Romero 37973

Docente: Carlos Andres Gomez

1. Introducción

Después de instalar y configurar Git, el siguiente paso consiste en aprender a administrar un repositorio y registrar las modificaciones de los archivos. Un repositorio contiene los archivos del proyecto y los datos necesarios para conservar su historial de versiones.

Git permite identificar qué archivos han cambiado, preparar las modificaciones que se desean registrar y crear confirmaciones que representan estados del proyecto. Estos procedimientos son fundamentales para mantener una documentación ordenada y recuperar información cuando sea necesario.

Este capítulo presenta los comandos esenciales para crear y clonar repositorios, agregar documentos, confirmar cambios y consultar el historial. Los ejemplos están orientados a una actividad individual con archivos de texto.

2. Objetivos

2.1. Objetivo general

Aplicar los comandos fundamentales de Git para administrar repositorios y registrar modificaciones en archivos de texto.

2.2. Objetivos específicos

- Crear y clonar repositorios.
- Identificar el estado de los archivos.
- Preparar modificaciones mediante git add.
- Registrar cambios mediante git commit.
- Consultar diferencias e historial de confirmaciones.
- Redactar mensajes de confirmación claros y descriptivos.

3. Desarrollo teórico

3.1. ¿Qué es un repositorio?

Un repositorio Git es una estructura que almacena los archivos del proyecto y los datos de su historial. Puede encontrarse en una computadora local o en un servidor remoto.

Existen dos procedimientos comunes para comenzar a trabajar: inicializar un directorio nuevo o clonar un repositorio existente.

Para inicializar un repositorio en la carpeta actual se utiliza:

git init

Este comando crea la estructura interna necesaria para comenzar a registrar versiones. No registra automáticamente todos los archivos existentes; es necesario agregarlos y confirmar los cambios.

Para obtener una copia de un repositorio alojado en GitHub se utiliza:

git clone https://github.com/USUARIO/REPOSITORIO.git

Se debe sustituir la dirección de ejemplo por la URL real del repositorio.

3.2. El estado de los archivos

Git distingue diferentes estados en el flujo de trabajo. Un archivo puede estar sin seguimiento, modificado o preparado para una confirmación.

El comando siguiente permite consultar el estado actual:

git status

Esta consulta ayuda a identificar archivos nuevos, modificaciones pendientes y cambios preparados para el siguiente commit.

Para observar las diferencias entre los archivos modificados y la última versión registrada se utiliza:

git diff

Este comando resulta útil antes de confirmar los cambios, porque permite revisar qué contenido se ha agregado o eliminado.

3.3. Agregar archivos con git add

El comando git add prepara los cambios que se desean incluir en la próxima confirmación. Esta etapa se conoce como área de preparación o *staging area*.

Para preparar un archivo específico:

git add documento1.txt

Para preparar todos los archivos nuevos y modificados que correspondan dentro del directorio actual:

git add .

Es recomendable revisar el resultado con git status antes de continuar.

3.4. Confirmar los cambios con git commit

Una confirmación registra en el historial los cambios que se encuentran preparados. Para crearla se utiliza:

git commit -m "docs: agregar documento inicial"

El texto que aparece después de -m es el mensaje de confirmación. Debe describir de forma breve el propósito del cambio.

Algunos ejemplos adecuados son:

- docs: agregar introduccion al tutorial
- docs: actualizar ejemplos de comandos
- fix: corregir errores de escritura
- feat: crear archivos de practica

Los prefijos son una convención útil, no un requisito obligatorio de Git. Lo importante es que el mensaje sea específico y comprensible.

3.5. Consultar el historial

Para revisar las confirmaciones realizadas se utiliza:

git log

El historial completo suele mostrar el identificador de cada commit, su autor, la fecha y el mensaje correspondiente.

Para obtener una versión resumida:

git log --oneline

Este comando facilita comprobar que los cambios se hayan registrado correctamente.

3.6. Diferencia entre add y commit

git add y git commit cumplen funciones distintas. El primero prepara los cambios que se incluirán en una confirmación; el segundo guarda esos cambios preparados en el historial local.

Por ello, editar un archivo no crea automáticamente una confirmación. Se debe revisar el estado, preparar los cambios y confirmar la versión.

4. Ejercicio práctico con archivos TXT

Se propone realizar un ejercicio usando un archivo llamado practica.txt.

Primero, se crea el archivo y se escribe el siguiente contenido:

Práctica de Git

Este archivo se utiliza para aprender

a registrar cambios mediante Git.

Dentro de la carpeta del repositorio se ejecuta:

git status

git add practica.txt

git status

git commit -m "docs: agregar archivo de practica"

git log --oneline

Después, se modifica el archivo y se agrega una nueva línea:

Git permite mantener un historial de versiones.

A continuación, se registran los cambios:

git diff

git add practica.txt

git commit -m "docs: ampliar contenido de practica"

git log --oneline

Se deben comparar los resultados de git status y git log --oneline antes y después de las operaciones. Es conveniente tomar capturas de pantalla para documentar el ejercicio.

5. Buenas prácticas

- Revisar los cambios antes de confirmarlos.
- Preparar solamente los archivos que correspondan al cambio.
- Evitar mensajes genéricos como cambios o prueba.
- Realizar confirmaciones pequeñas y relacionadas con una tarea concreta.
- Consultar el estado del repositorio antes y después de trabajar.
- No incluir contraseñas, tokens ni información privada en el repositorio.

6. Conclusiones

Los fundamentos de Git permiten registrar y organizar los cambios de un proyecto mediante un proceso controlado. Los comandos git status, git diff, git add, git commit y git log cumplen funciones complementarias que ayudan a revisar, preparar, confirmar y consultar las versiones de los archivos.

La elaboración de mensajes descriptivos mejora la comprensión del historial y facilita la identificación de las modificaciones realizadas durante la actividad.

7. Bibliografía

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.), capítulo 2: Fundamentos de Git. Apress. https://git-scm.com/book/es/v2/Fundamentos-de-Git

Git. (s. f.). *Obteniendo un repositorio Git*. https://git-scm.com/book/es/v2/Fundamentos-de-Git-Obteniendo-un-repositorio-Git

Git. (s. f.). *Documentación oficial de Git*. https://git-scm.com/doc